



Tarlac State University
College of Computer Studies
Bachelor of Science in Computer Science
Operating Systems



CPU Process Scheduling Algorithms

A Project

Submitted to the Faculty of the College of Computer Studies

San Isidro Campus, Tarlac City,

Tarlac State University

In Partial Fulfillment of the Requirements for the Course Operating Systems

Academic Year 2025-2026

Submitted By:

ROSACASE, Bei Fatima I.

Mrs. Jo Anne Cura

Subject Instructor

Date of submission: May 15, 2026

Table of Contents

- 1. Introduction4
 - 1.1 Project Overview4
 - Key Terminologies4
 - Types of CPU Scheduling5
 - 1.2 Project Purpose.....5
 - 1.3 Project Objectives.....5
- 2. Body6
 - 2.1 First Come, First Served (FCFS)6
 - 2.1.1 How It Works6
 - 2.1.2 Preemptive or Non-preemptive.....6
 - 2.1.3 Advantages and Limitations6
 - 2.1.4 Test Cases7
 - 2.2 Shortest Job First (SJF) — Non-preemptive.....8
 - 2.2.1 How It Works8
 - 2.2.2 Preemptive or Non-preemptive.....9
 - 2.2.3 Advantages and Limitations9
 - 2.2.4 Test Cases9
 - 2.3 Shortest Remaining Time (SRT) — Preemptive11
 - 2.3.1 How It Works11
 - 2.3.2 Preemptive or Non-preemptive.....11
 - 2.3.3 Advantages and Limitations12
 - 2.3.4 Test Cases12
 - 2.4 Round Robin (RR).....13
 - 2.4.1 How It Works13
 - 2.4.2 Preemptive or Non-preemptive.....14
 - 2.4.3 Advantages and Limitations14
 - 2.4.4 Test Cases14
 - 2.5 Priority Scheduling16
 - 2.5.1 How It Works16
 - 2.5.2 Preemptive or Non-preemptive.....16
 - 2.5.3 Advantages and Limitations17
 - 2.5.4 Test Cases17
 - 2.6 Priority Scheduling with Round Robin.....19
 - 2.6.1 How It Works19
 - 2.6.2 Preemptive or Non-preemptive.....19
 - 2.6.3 Advantages and Limitations19
 - 2.6.4 Test Cases20
- 3. Summary / Takeaways22
 - 3.1 Key Insights.....22
 - 3.2 Performance Differences Observed23
 - 3.3 Practical Relevance in Modern Operating Systems24
 - 3.4 Challenges and How They Were Addressed24
 - 3.5 Cross-Algorithm Comparison.....25
- 4. References26
- 5. Repository Link27

6. Source Code28

1. Introduction

1.1 Project Overview

Modern computing systems rely on the ability to execute multiple processes concurrently, and at the heart of this capability lies CPU scheduling — the mechanism that determines which process gains access to the processor at any given moment. As described by Silberschatz et al. (2021), scheduling is the most fundamental component of an operating system, directly influencing system throughput, responsiveness, and overall efficiency. Without a well-designed scheduling policy, even hardware-capable systems fail to utilize their resources productively.

CPU scheduling becomes particularly critical in multiprogrammed environments, where several processes reside in memory simultaneously. When one process blocks to wait for an I/O event, the CPU scheduler selects the next candidate from the ready queue and dispatches it, preventing the processor from remaining idle (Tanenbaum & Bos, 2023). This cycle — execute, block, reschedule — repeats thousands of times per second in a live system, and the cumulative effect of the scheduling choices made each cycle determines the end-user experience of the entire system.

The algorithms used to govern this selection process vary considerably in their assumptions and trade-offs. Some, like First-Come, First-Served (FCFS), prioritize simplicity and arrival order. Others, like Shortest Job First (SJF), aim to minimize average waiting time by always choosing the shortest available job. Preemptive algorithms like Shortest Remaining Time (SRT) go further by interrupting a running process whenever a shorter one arrives. Round Robin (RR) trades efficiency for fairness by cycling through processes in fixed time slices, while Priority Scheduling introduces urgency-based selection. Priority Scheduling with Round Robin combines both models to balance fairness and urgency (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

Understanding the behavioral characteristics of these algorithms through theoretical study alone is insufficient for the development of sound intuition about scheduling trade-offs. A hands-on simulation that makes the execution timeline visible — through Gantt charts and per-process metrics — provides the kind of experiential learning that bridges theory and practice (González-Rodríguez et al., 2024). This case study presents a web-based CPU Scheduling Simulator that implements all six algorithms mentioned above, accepts user-defined process parameters, and produces visual outputs that allow side-by-side comparison of their behavior.

Key Terminologies

The following terms are used consistently throughout this case study (Silberschatz et al., 2021; GeeksforGeeks, 2026):

- Arrival Time (AT) — The time at which a process enters the ready queue and becomes eligible for CPU execution.
- Burst Time (BT) — The total CPU time required by a process to complete its execution from start to finish.
- Completion Time (CT) — The time at which a process finishes its execution entirely.

- Turnaround Time (TAT) — The elapsed time from a process arrival to its completion. Computed as: $TAT = CT - AT$.
- Waiting Time (WT) — The total time a process spends waiting in the ready queue. Computed as: $WT = TAT - BT$.
- Time Quantum (TQ) — In Round Robin-based algorithms, the fixed CPU time slice allocated to each process per turn.
- Preemption — The act of forcibly removing a running process from the CPU before it completes, in favor of another process.

Types of CPU Scheduling

CPU scheduling is broadly classified into two types (Tanenbaum & Bos, 2023):

- Non-preemptive Scheduling — Once a process is assigned the CPU, it retains it until it either completes or voluntarily enters a waiting state. The scheduler cannot forcibly remove a running process.
- Preemptive Scheduling — The OS may interrupt a running process and reallocate the CPU to another process based on arrival of higher-priority or shorter-remaining-time processes.

1.2 Project Purpose

Despite the centrality of CPU scheduling in operating systems education, the concepts involved are notoriously difficult to internalize through lecture and reading alone. The abstract nature of process queues, context switches, and time-unit-by-time-unit state transitions makes it challenging for students to develop correct mental models of how algorithms behave under varied inputs. As noted by González-Rodríguez et al. (2024), interactive simulators that make scheduling behavior visible and measurable substantially improve student comprehension of these concepts.

This case study addresses that learning gap by developing a fully functional, web-based CPU scheduling simulator that allows a student to enter real process parameters, select an algorithm, and immediately observe the consequences through a dynamically generated Gantt chart and a summary table of per-process metrics. The simulator serves not only as a programming exercise but as an experimental environment where the learner can modify inputs, re-run simulations, and observe how scheduling decisions affect turnaround and waiting times across different process mixes.

By building this system from scratch, the developer also confronts implementation details that theoretical descriptions obscure — the edge cases that arise when processes share arrival times, the overhead introduced by frequent context switches in preemptive algorithms, and the precise accounting required to compute metrics correctly across varying execution sequences.

1.3 Project Objectives

This case study aims to design and develop a browser-based CPU scheduling simulator that demonstrates the behavior of six fundamental scheduling algorithms. The specific objectives are:

- To implement a functional, interactive web-based CPU scheduling simulator that accepts user-defined process parameters — including process identifiers, arrival times, burst times, and optional priority values — for a minimum of three (3) and a maximum of twenty (20) processes per simulation run.
- To correctly simulate six CPU scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF, Non-preemptive), Shortest Remaining Time (SRT, Preemptive), Round Robin (RR), Priority Scheduling (both Non-preemptive and Preemptive modes), and Priority Scheduling with Round Robin.
- To generate a dynamic Gantt chart for each simulation, accurately representing the process execution timeline including idle periods and preemption points.
- To compute and display per-process Waiting Time (WT) and Turnaround Time (TAT), as well as the Average Waiting Time and Average Turnaround Time for the complete process set.
- To expose the behavioral trade-offs between algorithms through side-by-side test cases so that differences in fairness, efficiency, and responsiveness become observable and measurable.

2. Body

2.1 First Come, First Served (FCFS)

2.1.1 How It Works

First-Come, First-Served is the most straightforward CPU scheduling algorithm: processes are executed in the exact order in which they arrive in the ready queue. The ready queue operates as a FIFO (First-In, First-Out) structure. When the CPU becomes free, the process at the front of the queue is dispatched, and it runs without interruption until it either completes or voluntarily enters a waiting state (Silberschatz et al., 2021). Any process that arrives while the CPU is occupied simply waits at the back of the queue without reordering.

FCFS is simple to implement and easy to reason about, but it is sensitive to process arrival order. If a process with a long burst time arrives first, all subsequent processes — regardless of how brief their own burst times are — must wait for the long process to finish. This behavior is known as the Convoy Effect and is the defining weakness of FCFS (Tanenbaum & Bos, 2023).

2.1.2 Preemptive or Non-preemptive

FCFS is strictly non-preemptive. Once the CPU is assigned to a process, that process retains the CPU until it completes execution. No other process can displace it during its run.

2.1.3 Advantages and Limitations

Advantages:

- Easy to implement due to its straightforward FIFO structure with no reordering logic required.
- Every process is eventually served; no process experiences indefinite postponement under a stable workload.
- Low scheduling overhead, as no priority comparisons or burst-time estimations are needed.

Limitations:

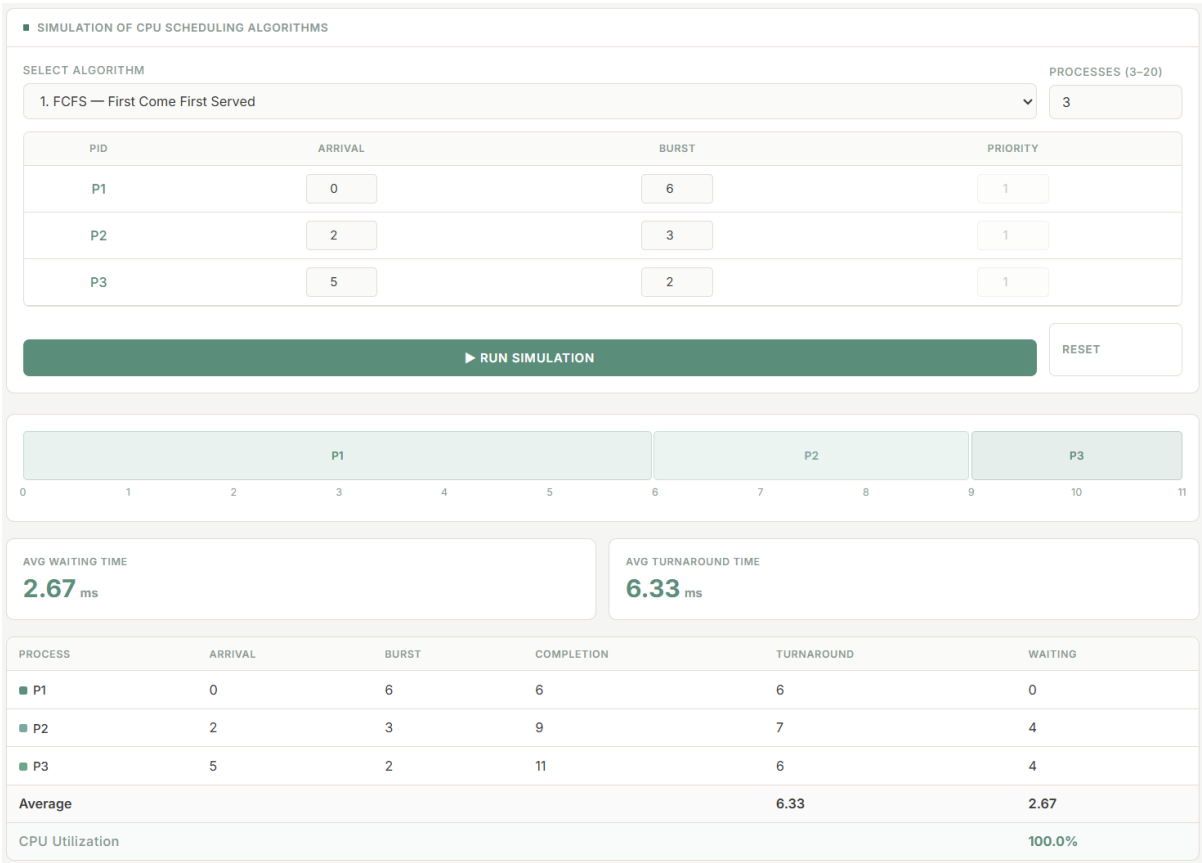
- Susceptible to the Convoy Effect: a long-burst process at the head of the queue forces shorter processes to wait far longer than necessary, inflating average waiting times for the entire process set (Silberschatz et al., 2021).
- Provides no guarantee of fast response time; unsuitable for interactive or time-sharing systems where responsiveness is critical.
- Ignores burst time and urgency, making it inefficient in mixed workloads where process lengths vary significantly.

2.1.4 Test Cases

Table 1. FCFS Test Case 1

Process	Arrival Time	Burst Time
P1	0	6
P2	2	3
P3	5	2

Figure 1. FCFS Test Case 1

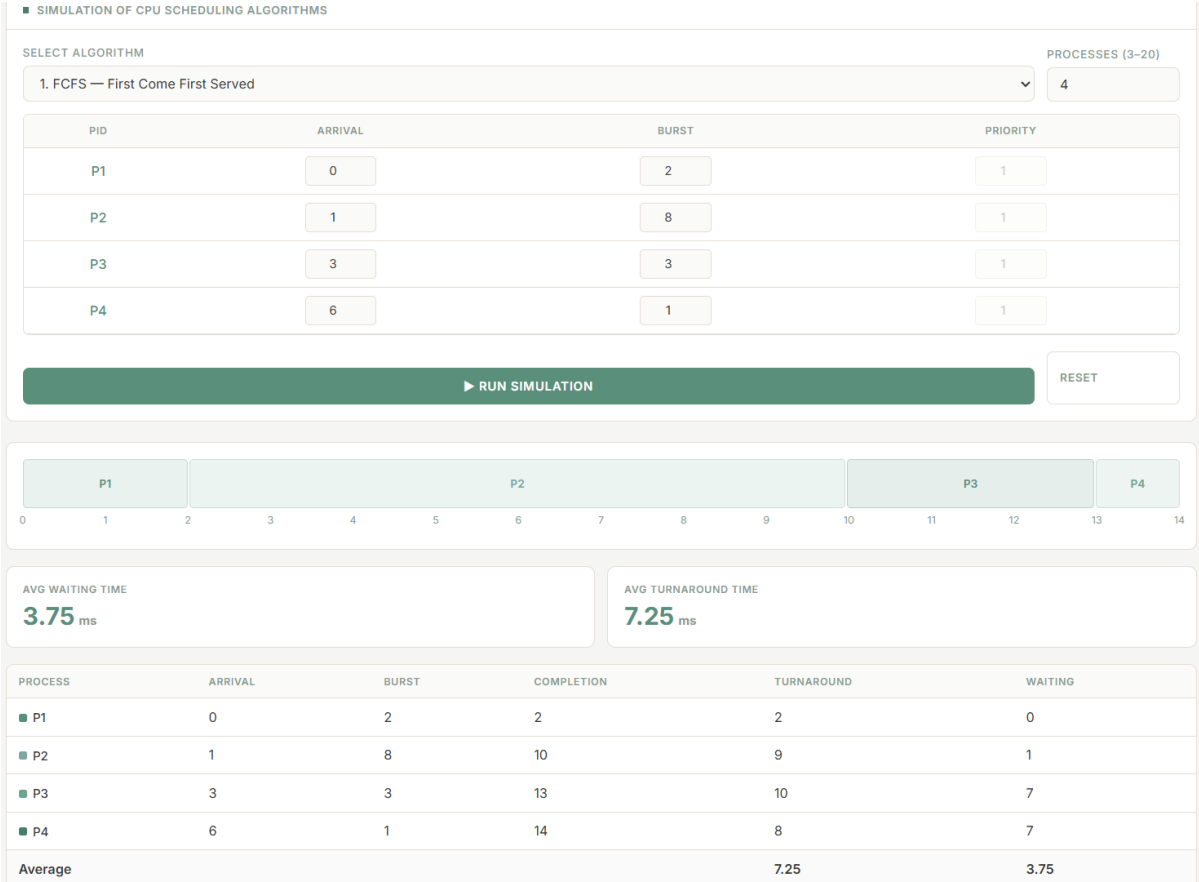


P1 occupies the CPU for its full burst of 6 units. P2, which arrived at time 2, must wait until time 6 before it can begin. P3 follows immediately after P2. The test illustrates that even when processes arrive before the current process finishes, they queue behind it without affecting execution order.

Table 2. FCFS Test Case 2

Process		Arrival Time	Bu
P1		0	
P2		1	
P3		3	
P4		6	

Figure 2. FCFS Test Case 2



This test demonstrates the Convoy Effect clearly. P2 has a burst of 8 and arrives at time 1, yet because P1 started first, P2 begins at time 2 and occupies the CPU until time 10. P3 and P4 arrived much earlier than their actual execution times.

2.2 Shortest Job First (SJF) Non-preemptive

2.2.1 How It Works

Shortest Job First selects the process with the smallest burst time from the set of currently ready processes whenever the CPU becomes available. It is a greedy algorithm: at every scheduling decision point, it makes the locally optimal choice of dispatching the shortest job, which can be proven to minimize average waiting time for a given batch of processes (Silberschatz et al., 2021). When two or more ready processes share the same burst time, a tie is broken by arrival time, and further ties by process index.

Unlike FCFS, SJF reorders the ready queue at each scheduling decision, bypassing longer processes in favor of shorter ones. This makes it well-suited for batch environments

where job lengths are known in advance, though impractical in general-purpose systems where burst times can only be estimated (Tanenbaum & Bos, 2023).

2.2.2 Preemptive or Non-preemptive

The variant implemented in this simulator is non-preemptive. Once the CPU is assigned to the selected process, it runs uninterrupted to completion, even if a shorter process arrives during execution.

2.2.3 Advantages and Limitations

Advantages:

- Produces the minimum possible average waiting time among all non-preemptive algorithms for a fixed set of processes when burst times are known (Silberschatz et al., 2021).
- Effectively eliminates the Convoy Effect by bypassing long processes in favor of shorter ones.
- Straightforward to implement in batch scheduling environments where execution times are predetermined.

Limitations:

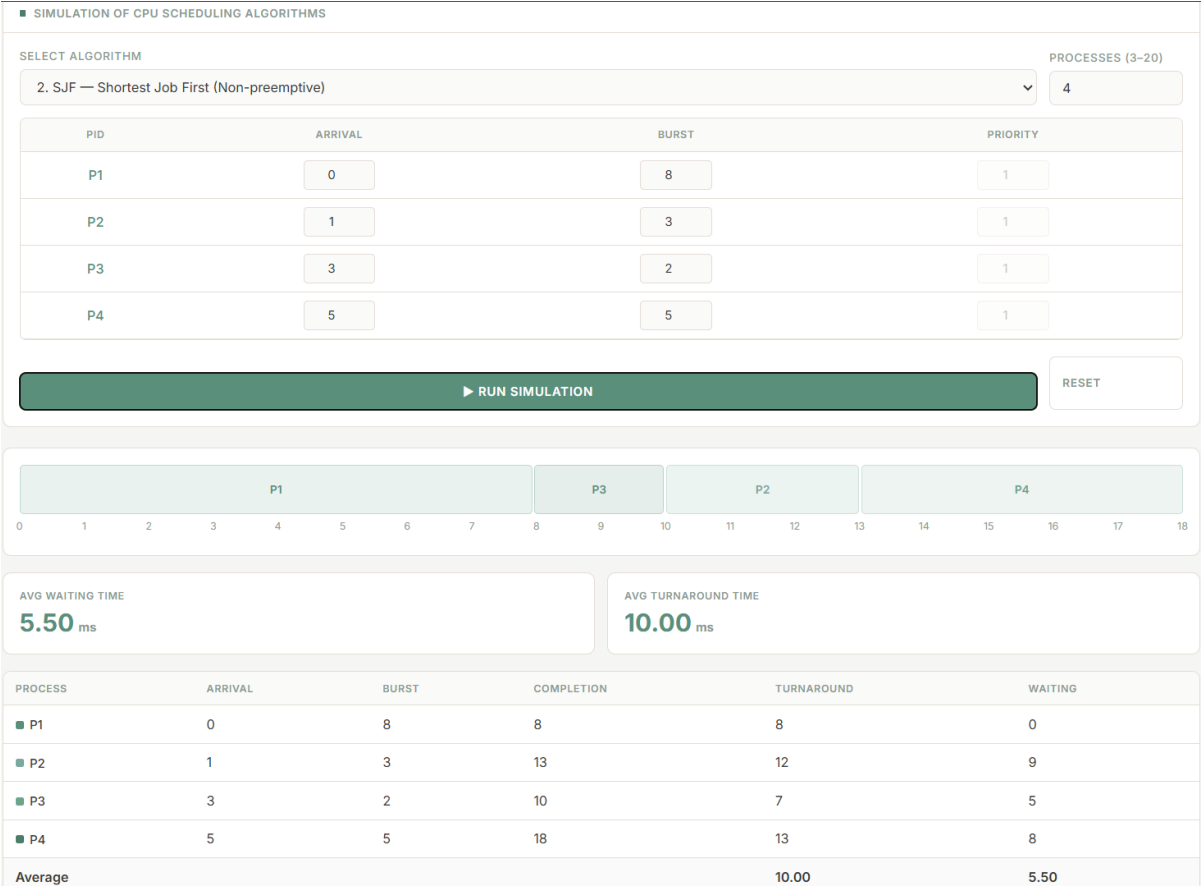
- Long processes may experience indefinite starvation if short processes continuously arrive and are always selected ahead of them (Arpaci-Dusseau & Arpaci-Dusseau, 2023).
- Requires accurate advance knowledge of burst times, which is generally unavailable in real interactive systems.
- Does not account for process urgency or priority; a critical but lengthy process is treated identically to a non-critical long process.

2.2.4 Test Cases

Table 3. SJF Test Case 1

Process	Arrival Time	Burst Time
P1	0	8
P2	1	3
P3	3	2
P4	5	5

Figure 3. SJF Test Case 1

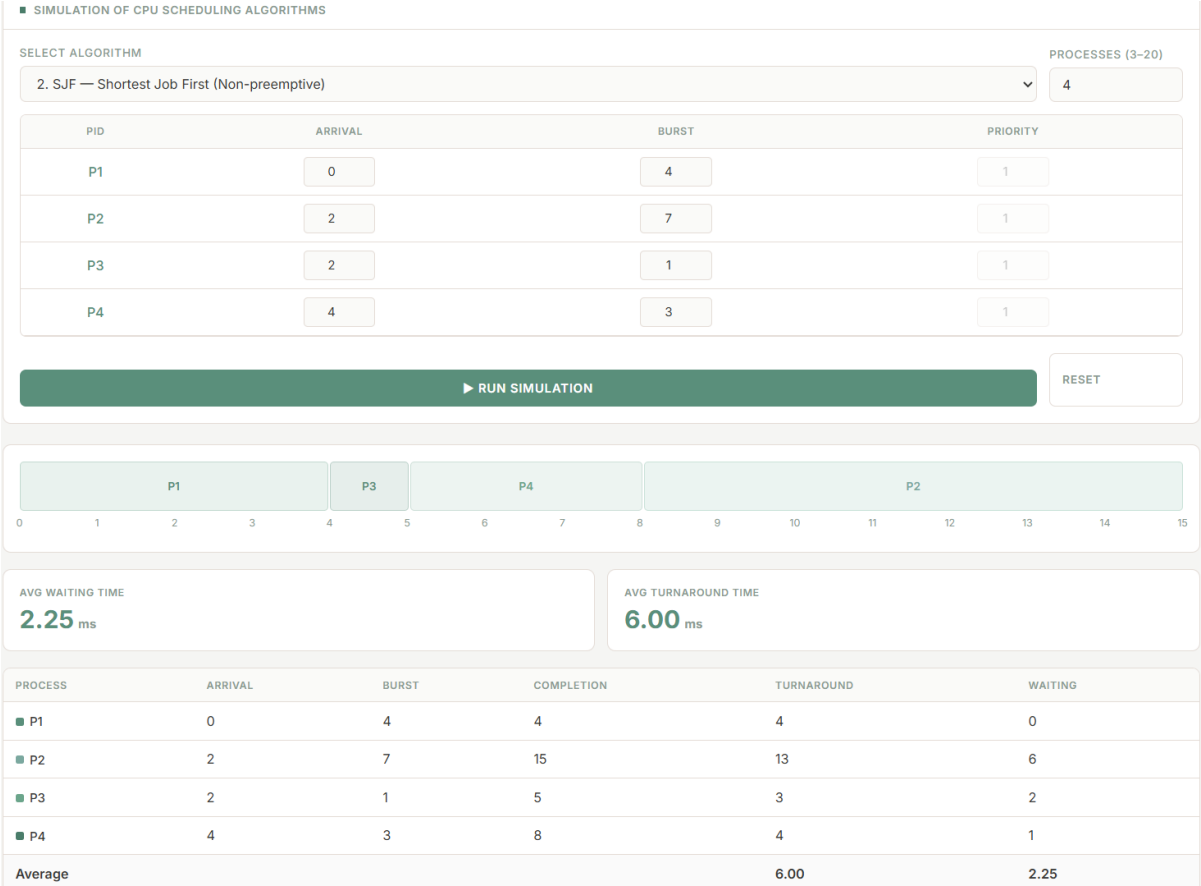


P1 is the only process at time 0 and runs to completion. At time 8, all remaining processes have arrived. The scheduler selects P3 (burst 2) first, then P2 (burst 3), and finally P4 (burst 5), demonstrating SJF reordering. At time 8, SJF compares all ready processes and selects the smallest burst time first.

Table 4. SJF Test Case 2

Process	Arrival Time	Burst Time
P1	0	4
P2	2	7
P3	2	1
P4	4	3

Figure 4. SJF Test Case 2



P1 starts at time 0. When it finishes at time 4, three processes are ready (P2, P3, P4). The scheduler picks P3 (burst 1), then P4 (burst 3), and finally P2 (burst 7). This demonstrates how SJF reduces the Convoy Effect compared to FCFS.

2.3 Shortest Remaining Time (SRT) — Preemptive

2.3.1 How It Works

Shortest Remaining Time is the preemptive variant of SJF. Instead of selecting the process with the shortest original burst time once at scheduling-decision points, SRT continuously monitors the remaining execution time of all ready processes. Whenever a new process arrives, the scheduler compares its burst time to the remaining time of the currently executing process. If the newcomer has a shorter remaining time, it immediately preempts the current process, which is returned to the ready queue with its updated remaining burst time (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

This constant attention to remaining burst times ensures that the process closest to completion is always executing, which is the defining property of the algorithm. SRT achieves the theoretical minimum average waiting time among all online scheduling algorithms when burst times are known, making it an important benchmark for comparison (Silberschatz et al., 2021).

2.3.2 Preemptive or Non-preemptive

SRT is fully preemptive. Every new process arrival triggers a comparison of remaining burst times, and the running process may be displaced at any time unit during its execution.

2.3.3 Advantages and Limitations

Advantages:

- Achieves the minimum average waiting time of all online algorithms when burst times are known (Silberschatz et al., 2021).
- Short processes that arrive late receive immediate service if their burst time is less than the running process remaining time, leading to fast turnaround for short tasks.
- More responsive than non-preemptive SJF in mixed workloads where short-burst jobs arrive unpredictably.

Limitations:

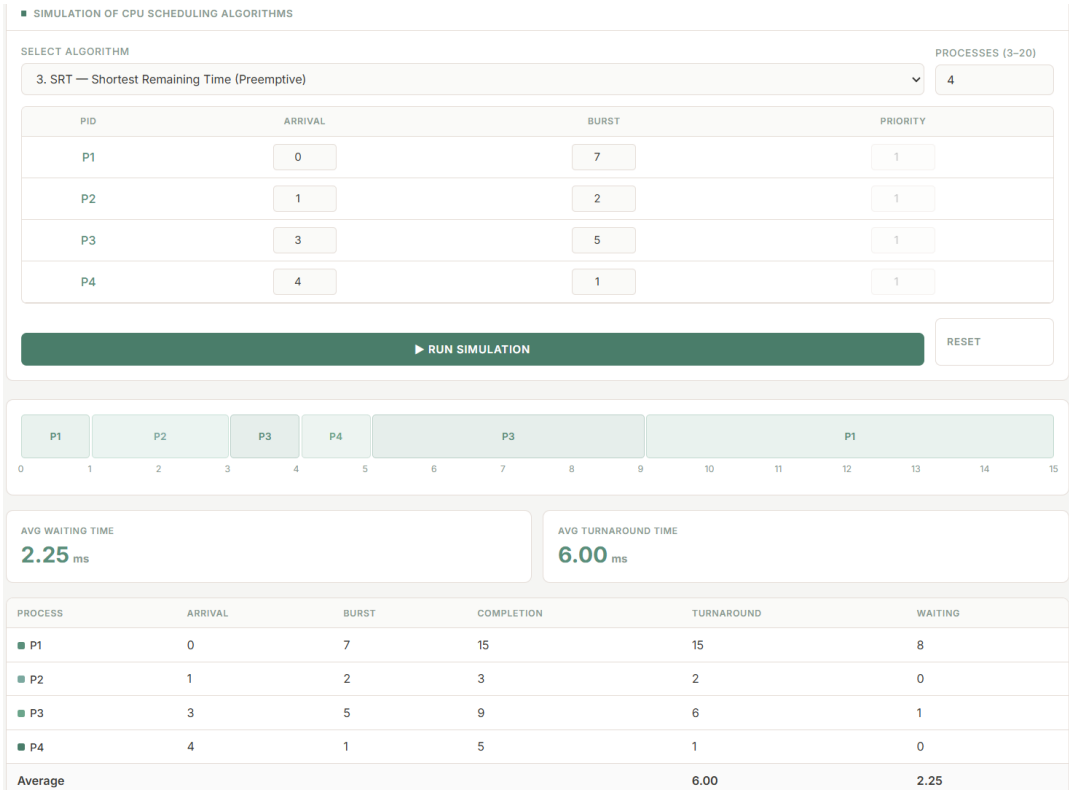
- Long processes are susceptible to severe starvation in systems where short processes arrive continuously (Tanenbaum & Bos, 2023).
- Generates significantly higher context-switch overhead compared to non-preemptive algorithms, which partially offsets its waiting-time advantage in practice.
- Requires accurate burst-time estimates, which are impractical in interactive general-purpose systems.

2.3.4 Test Cases

Table 5. SRT Test Case 1

Process	Arrival Time	Burst Time
P1	0	7
P2	1	2
P3	3	5
P4	4	1

Figure 5. SRT Test Case 1

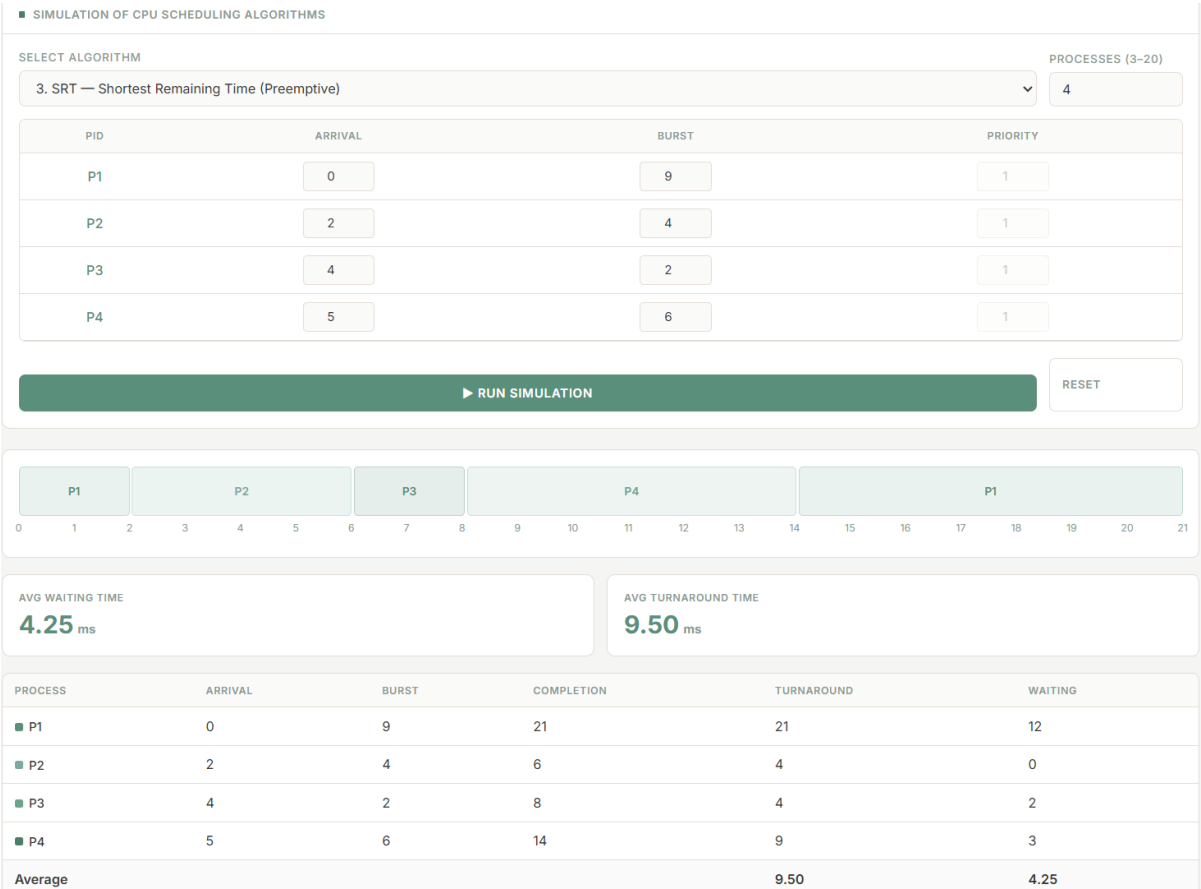


P1 begins at time 0 but is preempted at time 1 by P2 (remaining $2 < P1$ remaining 6). At time 3, P3 runs immediately but is preempted by P4. After P4 completes, P3 runs uninterrupted until P1 is last. Multiple preemptions are clearly visible.

Table 6. SRT Test Case 2

Process	Arrival Time	Burst Time
P1	0	9
P2	2	4
P3	4	2
P4	5	6

Figure 6. SRT Test Case 2



P1 starts execution at time 0. At time 2, P2 arrives with a shorter burst time than P1 remaining time, so P1 is preempted. At time 4, P3 arrives with an even shorter burst time and immediately preempts P2. Although P4 arrives at time 5, its burst time is longer than the remaining times of the currently ready processes, so it must wait. After P3 completes, P2 resumes and finishes, followed by P4, while P1 executes last because it accumulated the longest remaining time due to repeated preemptions.

2.4 Round Robin (RR)

2.4.1 How It Works

Round Robin assigns CPU time to processes in fixed-duration time slices, called time quanta, cycling through the ready queue in FIFO order. Each process executes for at most

one quantum before being returned to the back of the ready queue, where it waits for its next turn. If a process completes within its quantum, it exits the system and the next process is dispatched immediately (Silberschatz et al., 2021). Newly arrived processes are enqueued at the tail of the ready queue and receive CPU time on their first cycle.

The choice of time quantum strongly influences algorithm behavior. A very small quantum approximates a perfectly interleaved execution but increases context-switch overhead. A very large quantum allows processes to run longer per turn, reducing context switches but degrading responsiveness. When the quantum equals or exceeds the maximum burst time in the process set, Round Robin degenerates into FCFS (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

2.4.2 Preemptive or Non-preemptive

Round Robin is preemptive: a running process is forcibly removed after its time quantum expires, even if it has not completed, and placed at the back of the ready queue. Processes shorter than one quantum are not preempted but complete and exit normally.

2.4.3 Advantages and Limitations

Advantages:

- Provides equitable CPU sharing: every process is guaranteed CPU time within each full cycle, preventing starvation (Silberschatz et al., 2021).
- Delivers predictable and consistent response times, making it well-suited for interactive and time-sharing systems.
- Simple to implement once a FIFO queue and a timer mechanism are in place.

Limitations:

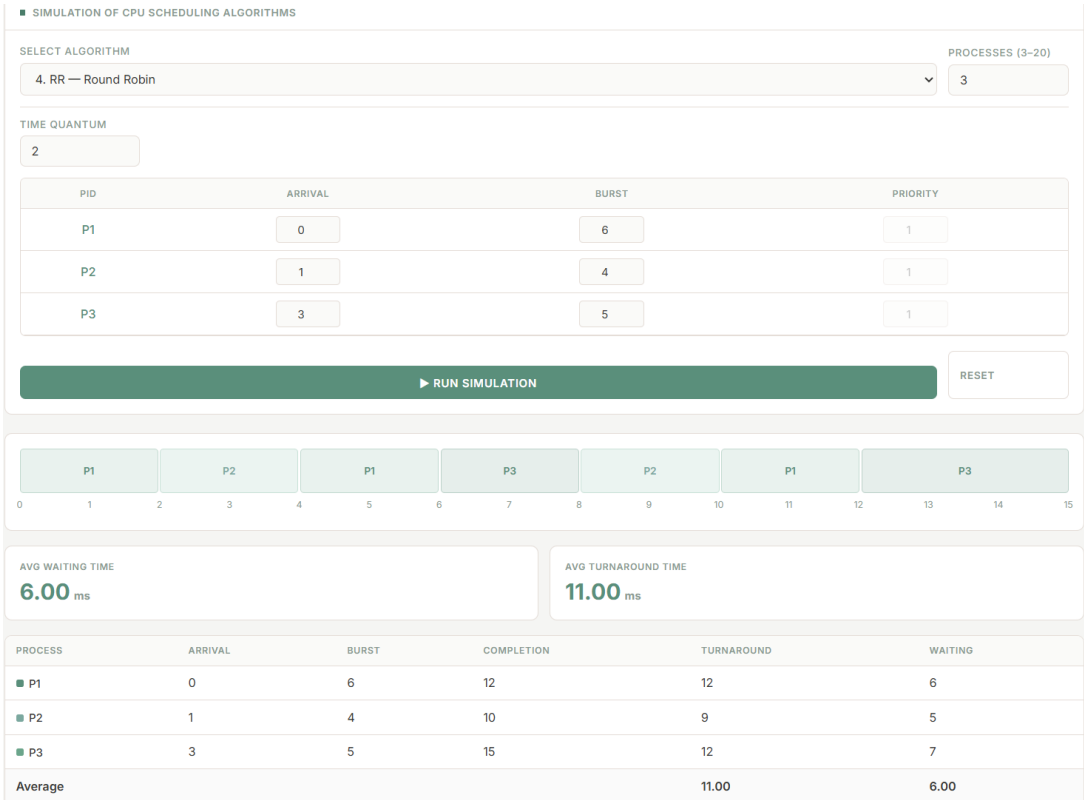
- Average waiting time and turnaround time are generally higher than SJF or SRT for mixed workloads, as processes cycle repeatedly before completing (Tanenbaum & Bos, 2023).
- Context-switch overhead accumulates rapidly with small time quanta, consuming CPU time that could otherwise be used for productive work.
- Selecting an appropriate time quantum requires knowledge of the workload; a poor choice can significantly degrade performance.

2.4.4 Test Cases

Table 7. RR Test Case 1 (TQ = 2)

Process	Arrival Time	Burst Time
P1	0	6
P2	1	4
P3	3	5

Figure 7. RR Test Case 1 (TQ = 2)

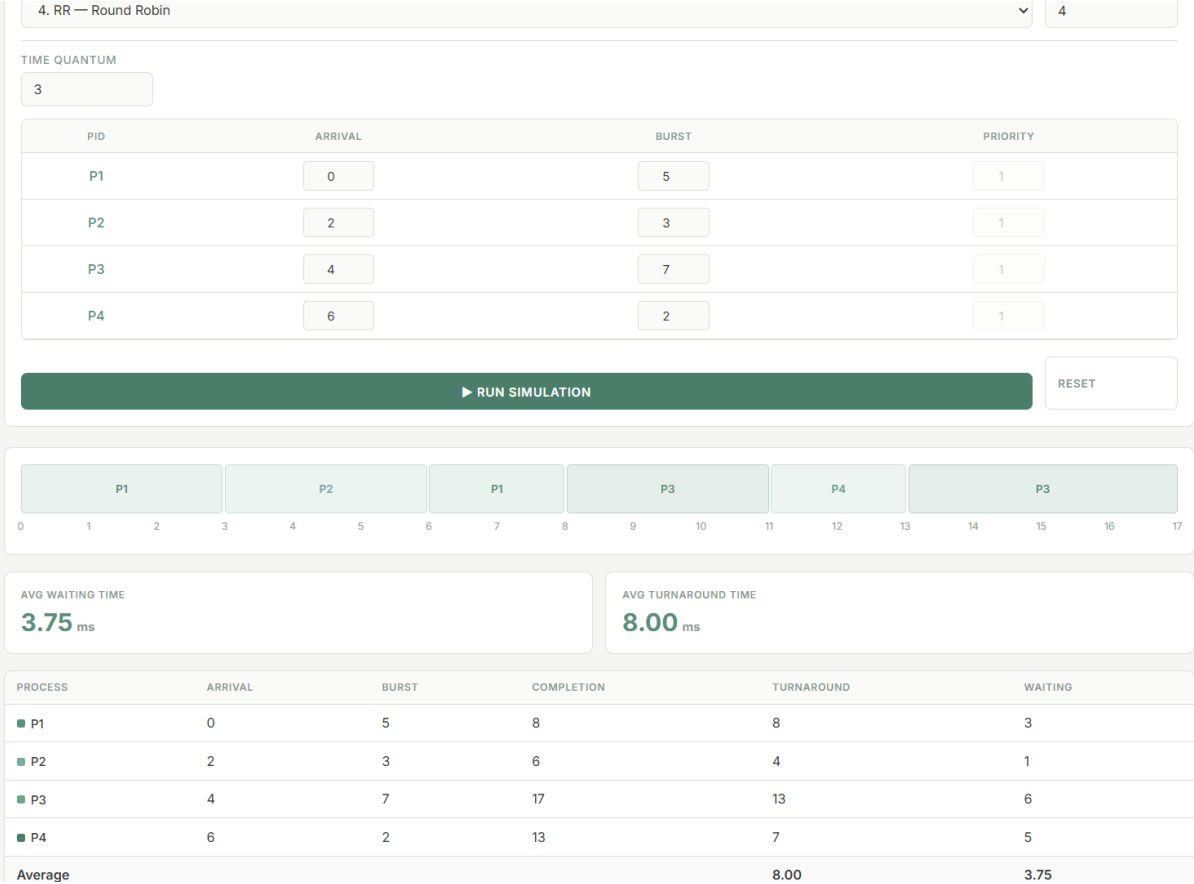


Using a time quantum of 2, each process receives at most two time units before being moved to the back of the ready queue if it still has remaining burst time. P1 begins first, followed by P2. P3 arrives at time 3 and joins the rotation during the succeeding cycle. The CPU repeatedly cycles among the processes until all bursts are completed.

Table 8. RR Test Case 2 (TQ = 3)

Process	Arrival Time	Burst Time
P1	0	5
P2	2	3
P3	4	7
P4	6	2

Figure 8. RR Test Case 2 (TQ = 3)



With a larger quantum of 3, P2 (burst 3) completes in exactly one quantum without being preempted, behaving identically to FCFS within its assigned slot. P3 (burst 7) requires two turns to complete. P4 (burst 2) finishes within a single quantum as well.

2.5 Priority Scheduling

2.5.1 How It Works

Priority Scheduling assigns each process a priority value and selects the highest-priority process from the ready queue whenever the CPU is available. The convention used in this simulator is configurable: users may specify either that a higher numeric value indicates higher priority, or that a lower numeric value indicates higher priority. When two or more processes share the same priority, they are dispatched in FCFS order (Silberschatz et al., 2021).

Priority Scheduling can be operated in two modes. In non-preemptive mode, the currently executing process runs to completion regardless of the arrival of higher-priority processes during its execution. In preemptive mode, a newly arriving process with higher priority immediately displaces the running process, which is returned to the ready queue with its remaining burst time preserved (Arpaci-Dusseau & Arpaci-Dusseau, 2023). Both modes are implemented in this simulator and selected by the user at runtime.

2.5.2 Preemptive or Non-preemptive

This simulator implements both variants. The user selects between Non-preemptive Priority Scheduling and Preemptive Priority Scheduling before running a simulation.

2.5.3 Advantages and Limitations

Advantages:

- Directly reflects the urgency or importance of processes, enabling system designers to guarantee timely execution of critical tasks (Silberschatz et al., 2021).
- The preemptive variant ensures that newly arriving high-priority processes are not delayed behind lower-priority ones, which is essential in real-time and safety-critical systems.
- Flexible and adaptable: priority values can encode any system-defined criterion, including deadline proximity, resource requirements, or user-assigned importance.

Limitations:

- Lower-priority processes are at constant risk of starvation, particularly in systems where high-priority processes arrive frequently (Tanenbaum & Bos, 2023).
- Determining appropriate priority values for diverse process types is non-trivial and application-dependent.
- In preemptive mode, frequent context switches for high-priority arrivals increase scheduling overhead.

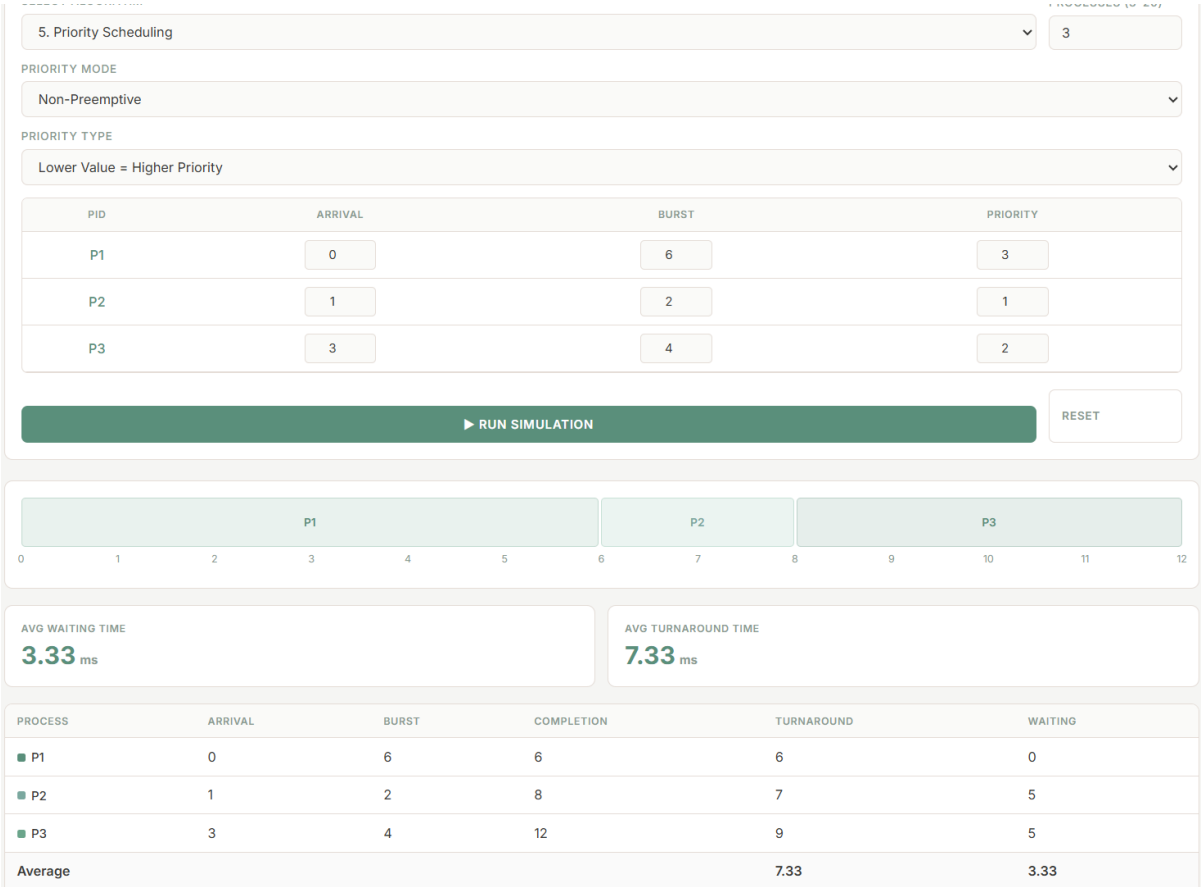
2.5.4 Test Cases

Note: In Test Case 1, lower numeric value = higher priority (Non-preemptive). In Test Case 2, higher numeric value = higher priority (Preemptive).

Table 9. Priority Test Case 1 — Lower Value = Higher Priority, Non-preemptive

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	2	1
P3	3	4	2

Figure 9. Priority Test Case 1

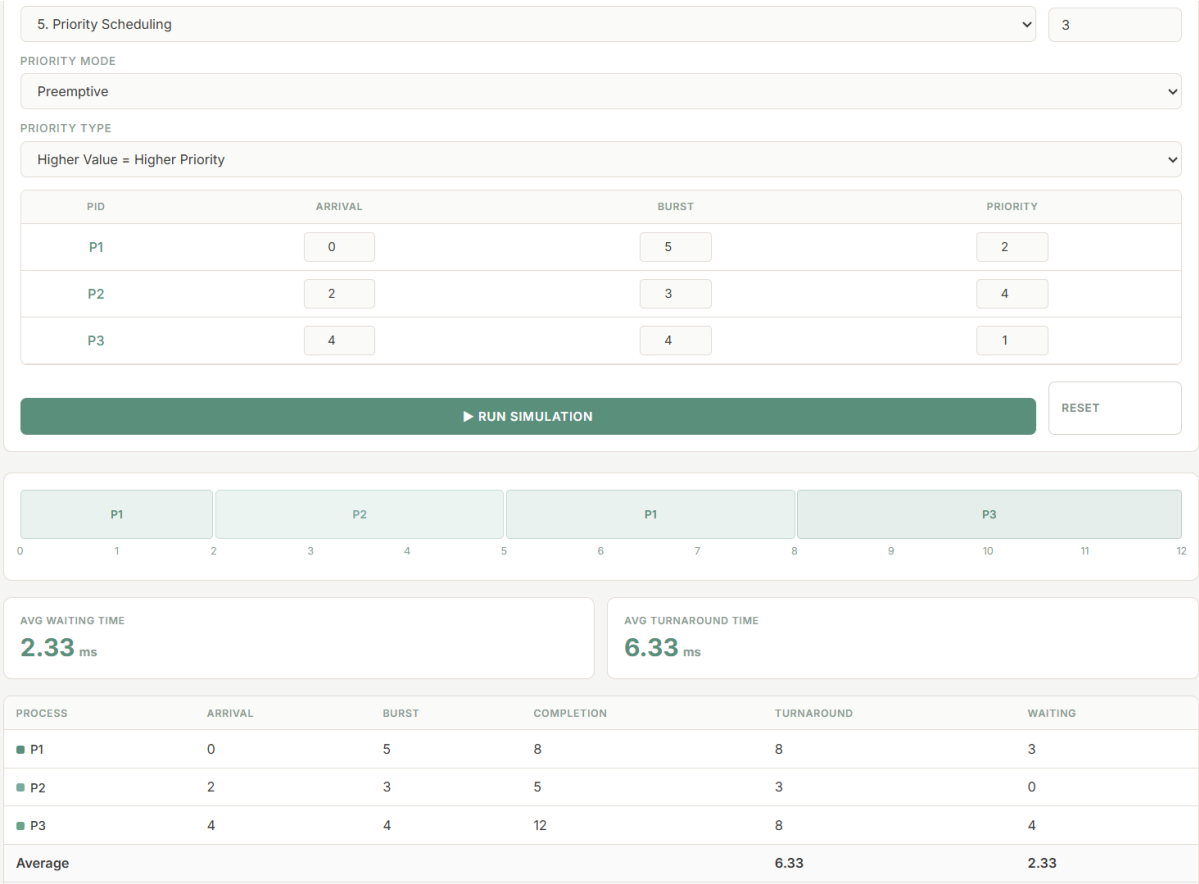


P1 starts at time 0 as the only available process. In non-preemptive mode, it runs to completion despite P2 arriving at time 1 with a higher priority. Once P1 finishes at time 6, P2 (priority 1, highest) is selected, followed by P3 (priority 2).

Table 10. Priority Test Case 2 — Higher Value = Higher Priority, Preemptive

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	2	3	4
P3	4	4	1

Figure 10. Priority Test Case 2 (Preemptive)



P1 starts execution at time 0. When P2 arrives at time 2 with a higher priority value under the higher-number-higher-priority convention, it immediately preempts P1. P2 continues executing because no arriving process has a higher priority than it. P3 arrives at time 4 with the lowest priority among the processes, so it cannot interrupt P2. After P2 finishes, P1 resumes and completes its remaining execution before P3 finally runs. This test case clearly demonstrates how preemptive priority scheduling favors higher-priority arrivals.

2.6 Priority Scheduling with Round Robin

2.6.1 How It Works

Priority Scheduling with Round Robin is a hybrid algorithm that layers time-quantum-based cycling on top of priority-based selection. Processes are first grouped and compared by their priority values, and the highest-priority group is always served before lower-priority groups. However, within a priority group, processes are not run to completion; instead, they cycle through Round Robin turns using the configured time quantum. This design ensures that high-priority processes receive preferential treatment while preventing any single high-priority process from monopolizing the CPU indefinitely (GeeksforGeeks, 2025).

When a newly arriving process has a higher priority than the currently executing process, it preempts the current process before its quantum expires, triggering an immediate priority re-evaluation of the ready queue. This combines preemptive priority selection with time-sliced fairness within priority groups.

2.6.2 Preemptive or Non-preemptive

Priority Scheduling with Round Robin is preemptive. A running process may be preempted both by quantum expiry and by the arrival of a higher-priority process.

2.6.3 Advantages and Limitations

Advantages:

- Eliminates indefinite starvation of same-priority processes by distributing CPU time fairly among processes at the same priority level (GeeksforGeeks, 2025).
- High-priority processes are still guaranteed preferential service over lower-priority ones, preserving the urgency-awareness of pure Priority Scheduling.
- Offers a practical middle ground between the strict urgency of Priority Scheduling and the fairness of Round Robin, making it suitable for diverse workloads.

Limitations:

- Lower-priority processes can still experience starvation if high-priority processes arrive in a continuous stream.
- The time quantum must be tuned carefully: too small increases context-switch overhead; too large degrades responsiveness for lower-priority groups.
- More complex to implement and reason about than either pure algorithm, requiring management of separate priority-group queues and remaining burst tracking simultaneously.

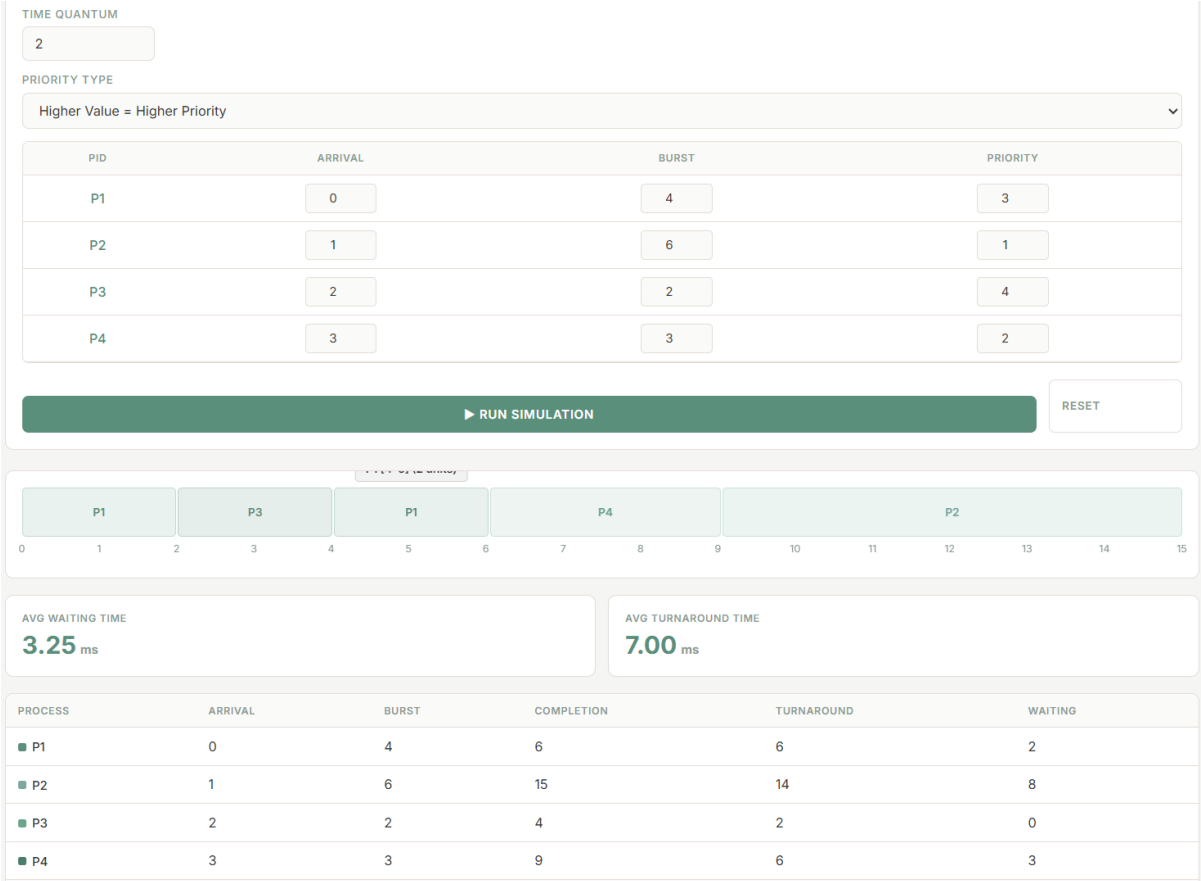
2.6.4 Test Cases

Note: In Test Case 1, higher value = higher priority. In Test Case 2, lower value = higher priority.

Table 11. Priority + RR Test Case 1 — Higher Value = Higher Priority (TQ = 2)

Process	Arrival Time	Burst Time	Priority
P1	0	4	3
P2	1	6	1
P3	2	2	4
P4	3	3	2

Figure 11. Priority + RR Test Case 1 (TQ = 2)

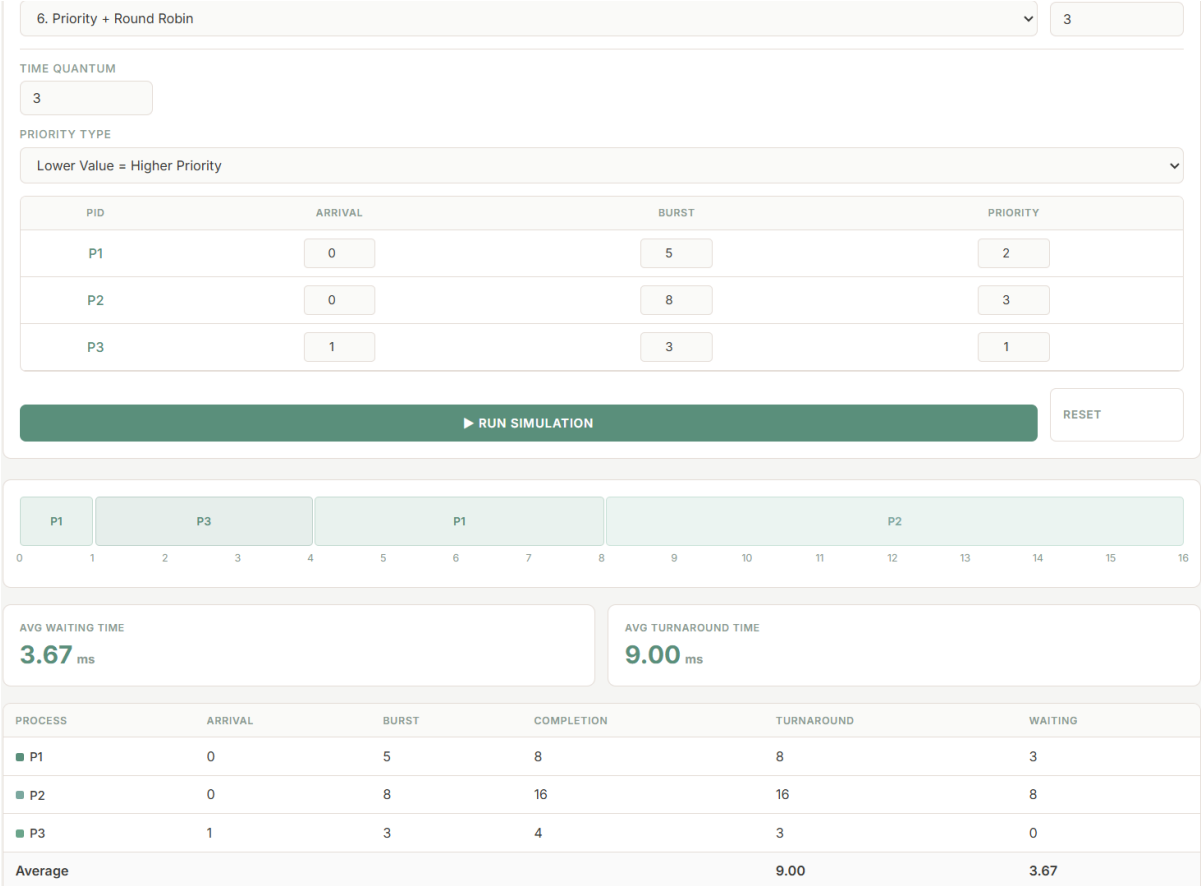


P3 (priority 4, highest) preempts P1 at time 2 upon arrival and runs for its full burst of 2. P1 (priority 3) then resumes and completes. P4 (priority 2) is next, followed by P2 (priority 1) which cycles through three quanta. The priority ordering is clearly reflected in execution sequence.

Table 12. Priority + RR Test Case 2 — Lower Value = Higher Priority (TQ = 3)

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	0	8	3
P3	1	3	1

Figure 12. Priority + RR Test Case 2 (TQ = 3)



P1 and P2 arrive together at time 0, but P1 executes first because it has the higher priority under the lower-number-higher-priority convention. When P3 arrives at time 1 with the highest priority value in this configuration, it immediately preempts the currently running process and executes to completion. After P3 finishes, P1 resumes and completes its remaining execution. P2, having the lowest priority, executes last and requires multiple quanta before finishing because its burst time exceeds the time quantum of 3. This test case demonstrates that higher-priority processes are always favored while Round Robin ensures fair CPU sharing among processes within the same priority level.

3. Summary / Takeaways

3.1 Key Insights

Developing this CPU scheduling simulator from the ground up offered a depth of understanding that reading about these algorithms in isolation cannot replicate. Several insights emerged from the implementation process that would not have been apparent from textbook descriptions alone.

First, the sensitivity of scheduling outcomes to small input variations was striking. A difference of one unit in arrival time, or a single unit of burst time, can fundamentally alter the execution order of all subsequent processes under preemptive algorithms. This sensitivity was most apparent in SRT and Preemptive Priority Scheduling, where a single new arrival could trigger a cascade of context switches that reshuffled the entire remaining schedule. What appears to be a minor change in the input table produces a visually and quantitatively different Gantt chart.

Second, correctly implementing preemption required a fundamentally different simulation approach than non-preemptive algorithms. Non-preemptive algorithms can be simulated at the granularity of complete process executions — select a process, advance time by its burst, record results. Preemptive algorithms require time to be advanced one unit at a time, with a full re-evaluation of the ready queue at every tick. Early attempts at implementing SRT by checking only at arrival events produced Gantt charts that were consistently wrong; the fix required restructuring the simulation loop to run at unit granularity throughout.

Third, the Priority Scheduling with Round Robin algorithm revealed how combining two independently reasonable policies can produce emergent complexity. Managing separate priority-group queues, tracking remaining burst times across multiple quantum boundaries, and handling preemption by higher-priority arrivals mid-quantum introduced implementation challenges that neither pure priority nor pure round-robin scheduling presented on their own.

3.2 Performance Differences Observed

Across all six algorithms and twelve test cases, clear patterns in performance characteristics emerged.

FCFS consistently produced the highest average waiting times in test cases where burst times varied significantly, a direct consequence of the Convoy Effect. In Test Case 2, a single process with burst time 8 delayed all subsequent processes by seven or more units each. FCFS performs well only when all processes have similar burst times, in which case the Convoy Effect does not manifest.

SJF produced substantially lower average waiting times than FCFS in every comparable test case, confirming its theoretical optimality for non-preemptive scheduling. However, the longer processes in SJF test cases accumulated high individual waiting times — the cost of favoring shorter processes. In workloads where process lengths are uniformly distributed, SJF advantage over FCFS diminishes.

SRT achieved the lowest average waiting times overall, as expected from its theoretical guarantee. However, the test cases also illustrated SRT liability: long processes that arrive early can be preempted repeatedly over many time units, accumulating waiting times far exceeding those they would experience under FCFS. P1 in SRT Test Case 2, for instance, waited ten units despite arriving first.

Round Robin demonstrated the most consistent and predictable per-process response times. Because every process receives CPU time within each cycle, no process waited excessively for its first execution turn. However, average turnaround times under RR were generally higher than SRT, as processes that could have completed quickly under SRT instead cycled through multiple quantum boundaries before finishing.

Priority Scheduling in non-preemptive mode behaved similarly to SJF in terms of structure, but with priority rather than burst time governing selection. The preemptive variant produced faster response for high-priority arrivals at the direct expense of lower-priority processes, which experienced longer waits whenever high-priority arrivals were frequent.

Priority Scheduling with Round Robin moderated this imbalance by ensuring that same-priority processes shared CPU time fairly.

3.3 Practical Relevance in Modern Operating Systems

The algorithms studied in this case study are not merely academic constructs — they form the conceptual foundation of scheduling strategies used in production operating systems today. The Linux kernel Completely Fair Scheduler (CFS) draws from the spirit of Round Robin and proportional-share scheduling, assigning virtual runtimes to processes and always selecting the process that has received the least CPU time relative to its weight (Linux Kernel Documentation, 2023). Windows employs a multilevel feedback queue that integrates both priority and time-quantum concepts, dynamically adjusting process priorities based on behavior (Microsoft, 2022).

Real-time operating systems used in aerospace, medical devices, and industrial control systems rely on preemptive priority scheduling to guarantee that critical tasks meet strict deadlines, directly reflecting the Priority Scheduling algorithm studied here. Understanding why these systems are designed the way they are — and what tradeoffs the designers accepted — requires precisely the kind of analytical foundation that this simulation exercise develops.

3.4 Challenges and How They Were Addressed

Several technical challenges arose during development.

The most significant was correctly implementing the preemptive algorithms, particularly SRT and Preemptive Priority Scheduling. Simulating time unit by unit — updating remaining burst times, checking for new arrivals, comparing candidates, and coalescing adjacent same-process blocks in the Gantt chart — required careful structuring of the simulation loop. Early versions failed to update the ready queue at every tick, producing incorrect results on test cases where a new arrival coincided with the current process remaining time shrinking below the newcomer burst time. The fix was a complete restructure of the inner simulation loop.

Generating the Gantt chart as a proportionally scaled HTML layout presented a separate challenge. Because the number of blocks and their relative widths depend on the algorithm and input, the chart must be built programmatically. Ensuring that adjacent blocks belonging to the same process were coalesced (so a process running for five consecutive units appeared as one wide block rather than five narrow ones) required a dedicated block-merging step in the rendering logic.

Input validation across six algorithms with different required parameters (some needing time quantum, some needing priority values, some needing preemption mode selection) was also more complex than anticipated. The solution was a unified validation function that checks only the fields relevant to the currently selected algorithm, applying visual error indicators to invalid inputs before allowing the simulation to proceed.

Finally, verifying the correctness of computed metrics required manually tracing each test case before entering it into the simulator, then comparing the simulator output against the manual trace. Several discrepancies were found and corrected during this process, each one pointing to a specific edge case in the algorithm logic.

3.5 Cross-Algorithm Comparison

Algorithm	Avg WT	Avg TAT	Starvation Risk	Preemptive?	Fairness
FCFS	High	High	None	No	High
SJF	Low	Low	High	No	Low
SRT	Lowest	Lowest	Severe	Yes	Low
Round Robin	Moderate	Moderate	None	Yes	Highest
Priority	Variable	Variable	High	Optional	Low
Priority + RR	Moderate	Moderate	Moderate	Yes	Moderate

No single algorithm is optimal for all workloads. FCFS is best when simplicity is paramount and burst times are similar. SJF and SRT are best when minimizing average waiting time is the primary goal and burst times are known. Round Robin is best for interactive systems requiring consistent responsiveness. Priority Scheduling is best when process urgency must be respected. Priority with Round Robin is best when both urgency and fairness must be balanced simultaneously.

4. References

- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). Operating systems: Three easy pieces (1.10 ed.). Arpaci-Dusseau Books. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- GeeksforGeeks. (2025, January 13). Priority scheduling in operating system. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/priority-scheduling-in-operating-system/>
- GeeksforGeeks. (2026, January 6). Round robin scheduling in operating system. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system/>
- GeeksforGeeks. (2026, January 7). Shortest job first or SJF CPU scheduling. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/shortest-job-first-or-sjf-cpu-scheduling/>
- GeeksforGeeks. (2026, April 24). CPU scheduling in operating systems. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/cpu-scheduling-in-operating-systems/>
- González-Rodríguez, M., Otero-Cerdeira, L., González-Rufino, E., & Rodríguez-Martínez, F. J. (2024). Study and evaluation of CPU scheduling algorithms. *Heliyon*, 10(9), e29959. <https://doi.org/10.1016/j.heliyon.2024.e29959>
- Linux Kernel Documentation. (2023). Completely fair scheduler. The Linux Kernel Archives. <https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html>
- Microsoft. (2022). Scheduling priorities. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/procthread/scheduling-priorities>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2021). Operating system concepts (10th ed.). John Wiley & Sons.
- Tanenbaum, A. S., & Bos, H. (2023). Modern operating systems (5th ed.). Pearson.
- Tutorialspoint. (n.d.). Round Robin (RR) scheduling algorithm. https://www.tutorialspoint.com/operating_system/os_round_robin_scheduling_algorithm.htm
- Wadmare, J., Kolte, D., Bhatia, K., Desai, P., & Wadmare, G. (2024). Virtual simulations tool for operating systems: Advancing e-learning in computing education. *Journal of Information Technology Education: Innovations in Practice*, 23, 018. <https://doi.org/10.28945/5404>

5. Repository Link

A working link to the program source code, video demonstration, and executable instructions is available at the following repository:

GitHub Repository: [ciaoski/CPU_Scheduling_Algorithm](#)

The repository contains:

- Video Demonstration (5–8 minutes with face-cam and voice-over)
- Source code (index.html with embedded CSS and JavaScript)
- Instructions on how to run the program (open index.html in any modern browser)

6. Source Code

The complete source code for this simulator is provided below. The program is implemented as a single HTML file with embedded CSS and JavaScript, requiring no build tools or server — it can be opened directly in any modern web browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>CPU Scheduler</title>
<style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    :root {
        --bg: #f5f5f3;
        --surface: #ffffff;
        --surface2: #fafaf8;
        --border: #e5ddd7;
        --accent: #5a8f7b;
        --accent2: #4a7d6a;
        --accent3: #6ba58b;
        --text: #3a3a38;
        --muted: #8b9a92;
        --hint: #7ba89f;
        --danger: #c72525;
        --sans: 'Inter', sans-serif;
    }

    body {
        font-family: var(--sans);
        background: var(--bg);
        color: var(--text);
        min-height: 100vh;
        display: flex;
        flex-direction: column;
        align-items: center;
    }

    header {
        width: 100%;
        padding: 28px 24px;
        display: flex;
        flex-direction: column;
        align-items: center;
        justify-content: center;
        border-bottom: 1px solid var(--border);
        background: var(--surface);
    }
    header h1 {
        font-family: var(--sans);
        font-size: 28px;
        font-weight: 600;
        letter-spacing: -0.5px;
        color: var(--text);
        text-transform: none;
    }

    .app {
        width: 100%;
        max-width: 1200px;
        padding: 24px 16px;
        display: flex;
        flex-direction: column;
        gap: 20px;
        align-items: center;
    }

    .input-wrapper { width: 100%; }
    .output-wrapper { width: 100%; }

    .panel {
        background: var(--surface);
        border: 1px solid var(--border);
    }
```

```
border-radius: 8px;
overflow: hidden;
width: 100%;
}
.panel-header {
padding: 12px 16px;
border-bottom: 1px solid var(--border);
display: flex;
align-items: center;
gap: 8px;
}
.panel-header span {
font-family: var(--sans);
font-size: 11px;
letter-spacing: 0.05em;
color: var(--muted);
text-transform: uppercase;
font-weight: 600;
}
.panel-icon {
width: 6px; height: 6px;
border-radius: 10%;
background: var(--accent2);
}
.panel-body { padding: 16px; }

label.field-label {
display: block;
font-size: 11px;
font-family: var(--sans);
color: var(--muted);
letter-spacing: 0.05em;
text-transform: uppercase;
margin-bottom: 6px;
font-weight: 500;
}

select, input[type="number"] {
width: 100%;
background: var(--surface2);
border: 1px solid var(--border);
```

```
color: var(--text);
font-family: var(--sans);
font-size: 13px;
padding: 8px 12px;
border-radius: 6px;
outline: none;
transition: border-color 0.15s, box-shadow 0.15s;
}
select:focus, input[type="number"]:focus {
border-color: var(--accent);
box-shadow: 0 0 0 2px rgba(90, 143, 123, 0.1);
}

.form-row { margin-bottom: 12px; }

.proc-table-wrap {
overflow-x: auto;
margin-top: 12px;
border: 1px solid var(--border);
border-radius: 6px;
}
.proc-table {
width: 100%;
border-collapse: collapse;
font-size: 13px;
}
.proc-table th {
font-family: var(--sans);
font-size: 10px;
letter-spacing: 0.05em;
text-transform: uppercase;
color: var(--muted);
padding: 10px 12px;
border-bottom: 1px solid var(--border);
text-align: center;
white-space: nowrap;
font-weight: 600;
background: var(--surface2);
}
.proc-table td {
padding: 8px 12px;
```

```

text-align: center;
border-bottom: 1px solid var(--border);
}

.proc-table td:first-child {
font-family: var(--sans);
font-size: 13px;
color: var(--accent);
font-weight: 500;
}

.proc-table input[type="number"] {
width: 70px;
padding: 6px 8px;
font-size: 12px;
text-align: center;
border-radius: 4px;
margin: 0 auto;
}

.proc-table td.priority-cell { opacity: 0.4; pointer-
events: none; transition: opacity 0.2s; }

.proc-table td.priority-cell.active { opacity: 1;
pointer-events: all; }

#tqRow { display: none; margin-top: 12px;
padding-top: 12px; border-top: 1px solid var(--
border); }

#tqRow.visible { display: block; }

.btn {
display: inline-flex;
align-items: center;
gap: 6px;
padding: 10px 20px;
font-family: var(--sans);
font-size: 12px;
letter-spacing: 0.05em;
border-radius: 6px;
border: none;
cursor: pointer;
transition: all 0.15s;
text-transform: uppercase;
font-weight: 600;
}

.btn-run {
background: var(--accent);
color: #ffffff;
width: 100%;
justify-content: center;
margin-top: 16px;
}

.btn-run:hover { background: var(--accent2); }
.btn-run:active { transform: scale(0.98); }

.btn-clear {
background: transparent;
border: 1px solid var(--border);
color: var(--muted);
font-size: 11px;
padding: 6px 12px;
}

.btn-clear:hover { border-color: var(--accent);
color: var(--text); }

.controls-row {
display: grid;
grid-template-columns: 1fr 180px;
gap: 12px;
align-items: end;
margin-bottom: 12px;
}

#errMsg {
display: none;
margin-top: 10px;
background: rgba(199, 37, 37, 0.08);
border: 1px solid rgba(199, 37, 37, 0.2);
color: #d32f2f;
border-radius: 6px;
padding: 8px 12px;
font-size: 12px;
}

.output-panel {
display: flex;
flex-direction: column;
gap: 16px;

```

```

}

.output-placeholder {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 80px 24px;
  gap: 16px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
}

.output-placeholder svg { opacity: 0.25; }

.output-placeholder p { color: var(--muted); font-size: 14px; font-family: var(--sans); }

.gantt-panel { background: var(--surface); border: 1px solid var(--border); border-radius: 8px; overflow: hidden; }

.gantt-scroll { overflow-x: auto; padding: 16px; }

.gantt-track { display: flex; align-items: stretch; min-height: 48px; gap: 2px; }

.gantt-block {
  display: flex;
  align-items: center;
  justify-content: center;
  font-family: var(--sans);
  font-size: 11px;
  font-weight: 600;
  border-radius: 4px;
  min-width: 28px;
  position: relative;
  cursor: default;
  transition: filter 0.1s;
}

.gantt-block:hover { filter: brightness(0.95); }
.gantt-block .g-label { pointer-events: none; }

.gantt-axis {
  display: flex;
  margin-top: 6px;
  font-family: var(--sans);
  font-size: 10px;
  color: var(--muted);
  position: relative;
}

.gantt-tick { position: absolute; transform: translateX(-50%); }

.pc0 { background: rgba(107,165,139,0.15); border: 1px solid rgba(107,165,139,0.3); color: #5a8f7b; }
.pc1 { background: rgba(122,184,159,0.15); border: 1px solid rgba(122,184,159,0.3); color: #7ba89f; }
.pc2 { background: rgba(90,143,123,0.15); border: 1px solid rgba(90,143,123,0.3); color: #5a8f7b; }
.pc3 { background: rgba(107,165,139,0.12); border: 1px solid rgba(107,165,139,0.25); color: #6ba58b; }
.pc4 { background: rgba(74,125,106,0.15); border: 1px solid rgba(74,125,106,0.3); color: #4a7d6a; }
.pc5 { background: rgba(122,184,159,0.12); border: 1px solid rgba(122,184,159,0.25); color: #7ba89f; }
.pc6 { background: rgba(90,143,123,0.18); border: 1px solid rgba(90,143,123,0.35); color: #5a8f7b; }
.pc7 { background: rgba(107,165,139,0.14); border: 1px solid rgba(107,165,139,0.28); color: #6ba58b; }
.pc8 { background: rgba(122,184,159,0.16); border: 1px solid rgba(122,184,159,0.32); color: #7ba89f; }
.pc9 { background: rgba(74,125,106,0.12); border: 1px solid rgba(74,125,106,0.25); color: #4a7d6a; }
.pc10 { background: rgba(90,143,123,0.14); border: 1px solid rgba(90,143,123,0.28); color: #5a8f7b; }
.pc11 { background: rgba(107,165,139,0.16); border: 1px solid rgba(107,165,139,0.32); color: #6ba58b; }
.pc12 { background: rgba(122,184,159,0.13); border: 1px solid rgba(122,184,159,0.26); color: #7ba89f; }

```

```

        .pc13 {
background:rgba(90,143,123,0.15);    border:1px
solid rgba(90,143,123,0.3); color:#5a8f7b; }

        .pc14 {
background:rgba(74,125,106,0.14);    border:1px
solid rgba(74,125,106,0.28); color:#4a7d6a; }

        .pc15 { background:rgba(107,165,139,0.13);
border:1px solid
rgba(107,165,139,0.26);color:#6ba58b; }

        .pc16 { background:rgba(122,184,159,0.15);
border:1px solid rgba(122,184,159,0.3);
color:#7ba89f; }

        .pc17 {
background:rgba(90,143,123,0.12);    border:1px
solid rgba(90,143,123,0.25); color:#5a8f7b; }

        .pc18 { background:rgba(107,165,139,0.17);
border:1px solid
rgba(107,165,139,0.34);color:#6ba58b; }

        .pc19 {
background:rgba(74,125,106,0.16);    border:1px
solid rgba(74,125,106,0.32); color:#4a7d6a; }

        .idle-block { background:rgba(160,160,160,0.08);
border:1px solid rgba(160,160,160,0.2);
color:#999; font-size:10px; font-style:italic; }

        .metrics-panel { background: var(--surface);
border: 1px solid var(--border); border-radius: 8px;
overflow: hidden; }

        .metrics-table { width: 100%; border-collapse:
collapse; font-size: 13px; }

        .metrics-table th {

font-family: var(--sans);

font-size: 10px;

letter-spacing: 0.05em;

text-transform: uppercase;

color: var(--muted);

padding: 10px 12px;

border-bottom: 1px solid var(--border);

text-align: left;

font-weight: 600;

background: var(--surface2);
}

        .metrics-table td {

padding: 10px 12px;

border-bottom: 1px solid var(--border);

font-size: 13px;
}

```

```

        .metrics-table tr:last-child td { border-bottom:
none; }

        .pid-badge { display: inline-flex; align-items:
center; gap: 6px; font-family: var(--sans); font-size:
12px; font-weight: 500; }

        .pid-dot { width: 8px; height: 8px; border-radius:
2px; flex-shrink: 0; }

        .stats-row { display: grid; grid-template-columns:
1fr 1fr; gap: 12px; }

        .stat-card { background: var(--surface); border:
1px solid var(--border); border-radius: 8px;
padding: 16px; }

        .stat-label { font-family: var(--sans); font-size:
10px; letter-spacing: 0.05em; color: var(--muted);
text-transform: uppercase; margin-bottom: 6px;
font-weight: 600; }

        .stat-value { font-family: var(--sans); font-size:
24px; font-weight: 700; color: var(--accent); }

        .stat-unit { font-size: 12px; color: var(--muted);
margin-left: 4px; }

        .algo-badge {

display: inline-flex;

align-items: center;

gap: 6px;

background: rgba(90,143,123,0.12);

border: 1px solid rgba(90,143,123,0.25);

color: var(--accent);

font-family: var(--sans);

font-size: 11px;

padding: 4px 12px;

border-radius: 20px;

letter-spacing: 0.05em;

font-weight: 500;
}

        .gantt-block:hover::after {

content: attr(data-tip);

position: absolute;

bottom: calc(100% + 6px);

left: 50%;

transform: translateX(-50%);

background: var(--surface);

border: 1px solid var(--border);
}

```

```
color: var(--text);
font-size: 11px;
white-space: nowrap;
padding: 6px 10px;
border-radius: 4px;
pointer-events: none;
z-index: 10;
}

@media (max-width: 750px) {
  .app { grid-template-columns: 1fr; }
  .stats-row { grid-template-columns: 1fr; }
}

</style>
</head>
<body>

<header>
  <h1>CPU Scheduler</h1>
</header>

<div class="app">

  <!-- INPUT PANEL -->
  <div class="input-wrapper">
    <aside>
      <div class="panel">
        <div class="panel-header">
          <div class="panel-icon"></div>
          <span>Simulation of CPU Scheduling Algorithms</span>
        </div>
        <div class="panel-body">

          <div style="display:grid;grid-template-columns:1fr 130px;gap:12px;align-items:end;margin-bottom:12px;">
            <div>
              <label class="field-label">Select Algorithm</label>
              <select id="algoSelect">
                <option value="">— select —</option>
```

```
                <option value="FCFS">1. FCFS — First Come First Served</option>
                <option value="SJF">2. SJF — Shortest Job First (Non-preemptive)</option>
                <option value="SRT">3. SRT — Shortest Remaining Time (Preemptive)</option>
                <option value="RR">4. RR — Round Robin</option>
                <option value="Priority">5. Priority Scheduling </option>
                <option value="PriorityRR">6. Priority + Round Robin</option>
              </select>
            </div>
            <div>
              <label class="field-label">Processes (3–20)</label>
              <input type="number" id="procCount" min="3" max="20">
            </div>
          </div>

          <!-- Time quantum -->
          <div id="tqRow" style="margin-bottom:12px;">
            <label class="field-label">Time Quantum</label>
            <input type="number" id="timeQuantum" min="1" placeholder="2" style="max-width:130px;">
          </div>

          <!-- Preemptive mode (Priority only) -->
          <div id="preemptiveRow" style="margin-bottom:12px;display:none;">
            <label class="field-label">Priority Mode</label>
            <select id="preemptiveSelect">
              <option value="false">Non-Preemptive</option>
              <option value="true">Preemptive</option>
            </select>
          </div>

          <!-- Priority direction -->
```



```

const errMsg =
document.getElementById('errMsg');

const outputPanel =
document.getElementById('outputPanel');

/* — FIX: single module-level variable, read once
per run — */

let higherPriority = true;

/* Color classes per process index (0-based) */

const PC =
Array.from({length:20}, (_,i) => `pc${i}`);

const COLORS =
['#5a8f7b', '#7ba89f', '#6ba58b', '#4a7d6a', '#5a8f7b',
  '#6ba58b', '#7ba89f', '#4a7d6a', '#5a8f7b', '#
7ba89f',
  '#6ba58b', '#4a7d6a', '#5a8f7b', '#6ba58b', '#
7ba89f',
  '#4a7d6a', '#5a8f7b', '#7ba89f', '#6ba58b', '#
4a7d6a'];

/* — UI helpers — */

function needsTQ(a) { return
a==='RR' || a==='PriorityRR'; }

function needsPri(a) { return
a==='Priority' || a==='PriorityRR'; }

function needsPreemptive(a) { return
a==='Priority'; }

algoSel.addEventListener('change', () => {
  const a = algoSel.value;

  tqRow.classList.toggle('visible', needsTQ(a));

  preemptiveRow.style.display =
needsPreemptive(a) ? 'block' : 'none';

  priorityTypeRow.style.display = needsPri(a) ?
'block' : 'none';

  document.querySelectorAll('#procBody .priority-
cell').forEach(td =>

    td.classList.toggle('active', needsPri(a))

  );
});

countInput.addEventListener('input', buildTable);

```

```

function buildTable() {
  const n = parseInt(countInput.value);

  if (!n || n < 3 || n > 20) { procBody.innerHTML =
"; return; }

  const a = algoSel.value;

  let html = "";

  for (let i = 0; i < n; i++) {

    const existingRow = procBody.rows[i];

    const at = existingRow ?
existingRow.cells[1].querySelector('input').value :
";

    const bt = existingRow ?
existingRow.cells[2].querySelector('input').value :
";

    const pr = existingRow ?
existingRow.cells[3].querySelector('input').value :
";

    html += `<tr>

      <td>P${i+1}</td>

      <td><input type="number" min="0"
placeholder="0" value="${at}"></td>

      <td><input type="number" min="1"
placeholder="" value="${bt}"></td>

      <td class="priority-cell${needsPri(a)?
'active':''}>

        <input type="number" min="1"
placeholder="1" value="${pr}">

      </td>

    </tr>`;

  }

  procBody.innerHTML = html;
}

function readProcesses() {
  return Array.from(procBody.rows).map((row, i)
=> {

    const inputs = row.querySelectorAll('input');

    return {

      id: `P${i+1}`,

      arrival: parseInt(inputs[0].value) || 0,

      burst: parseInt(inputs[1].value),

      priority: parseInt(inputs[2].value) || 1,

      idx: i

    };

  });
}

```

```

}

function      showErr(msg)      {
errMsg.style.display='block';
errMsg.textContent=msg; }

function      clearErr()        {
errMsg.style.display='none'; errMsg.textContent=""
; }

/*
=====
=====
=====

SCHEDULING ALGORITHMS

=====
=====
===== */

function clone(procs) { return procs.map(p=>({...p,
remaining:p.burst})); }

function pushBlock(timeline, pid, start, end) {
  if (end <= start) return;
  const last = timeline[timeline.length - 1];
  if (last && last.pid === pid && last.end === start) {
    last.end = end;
  } else {
    timeline.push({ pid, start, end });
  }
}

/* 1. FCFS */
function fcfs(procs) {
  const sorted = [...procs].sort((a,b)=>a.arrival-
b.arrival||a.idx-b.idx);
  const timeline = [], finish = {};
  let t = 0;
  for (const p of sorted) {
    if (t < p.arrival) {
      timeline.push({ pid:'idle', start:t, end:p.arrival });
      t = p.arrival;
    }
    timeline.push({ pid:p.id, start:t, end:t+p.burst });
    finish[p.id] = t + p.burst;
  }
}

```

```

    t += p.burst;
  }
  return buildMetrics(procs, timeline, finish);
}

/* 2. SJF */
function sjf(procs) {
  const ps = clone(procs);
  const done = new Set(), timeline = [], finish = {};
  let t = 0, completed = 0;
  while (completed < ps.length) {
    const ready = ps.filter(p=>p.arrival<=t &&
!done.has(p.id));
    if (!ready.length) {
      const next =
Math.min(...ps.filter(p=>!done.has(p.id)).map(p=>
p.arrival));
      if (t < next) { pushBlock(timeline, 'idle', t, next);
t = next; }
      else { t++; }
      continue;
    }
    ready.sort((a,b)=>a.burst-b.burst||a.arrival-
b.arrival||a.idx-b.idx);
    const p = ready[0];
    timeline.push({ pid:p.id, start:t, end:t+p.burst });
    finish[p.id] = t + p.burst;
    t += p.burst;
    done.add(p.id); completed++;
  }
  return buildMetrics(procs, timeline, finish);
}

/* 3. SRT */
function srt(procs) {
  const ps = clone(procs);
  const finish = {}, timeline = [];
  let t = 0, completed = 0;
  while (completed < ps.length) {
    const ready = ps.filter(p=>p.arrival<=t &&
p.remaining>0);
    if (!ready.length) {

```

```

        const next =
Math.min(...ps.filter(p=>p.remaining>0).map(p=>p
.arrival));

    pushBlock(timeline, 'idle', t, next);
    t = next; continue;
}

    ready.sort((a,b)=>a.remaining-
b.remaining||a.arrival-b.arrival||a.idx-b.idx);
    const p = ready[0];
    pushBlock(timeline, p.id, t, t+1);
    p.remaining--; t++;

    if (p.remaining === 0) { finish[p.id] = t;
completed++; }
}

    return buildMetrics(procs, timeline, finish);
}

/* 4. Round Robin */
function roundRobin(procs, tq) {
    const ps = clone(procs).sort((a,b)=>a.arrival-
b.arrival||a.idx-b.idx);

    const finish = {}, timeline = [];

    let t = 0, queue = [], enqueued = new Set(),
completed = 0;

    function enqueueArrivals() {
        ps.forEach(p => {
            if (p.arrival <= t && !enqueued.has(p.id) &&
p.remaining > 0) {
                queue.push(p); enqueued.add(p.id);
            }
        });
    }

    enqueueArrivals();

    while (completed < ps.length) {
        if (!queue.length) {
            const next = ps.filter(p=>p.remaining>0 &&
!enqueued.has(p.id))
                .reduce((m,p)=>p.arrival<m?p.arrival
:m, Infinity);
            if (next === Infinity) break;
            pushBlock(timeline, 'idle', t, next);

```

```

        t = next; enqueueArrivals(); continue;
    }

    const p = queue.shift();
    const slice = Math.min(tq, p.remaining);
    pushBlock(timeline, p.id, t, t+slice);
    p.remaining -= slice; t += slice;
    enqueueArrivals();

    if (p.remaining === 0) { finish[p.id] = t;
completed++; }
    else { queue.push(p); }
}

    return buildMetrics(procs, timeline, finish);
}

/* 5. Priority (non-preemptive) — uses module-
level `higherPriority` */
function priority(procs) {
    const ps = clone(procs);
    const done = new Set(), timeline = [], finish = {};
    let t = 0, completed = 0;

    while (completed < ps.length) {
        const ready = ps.filter(p=>p.arrival<=t &&
!done.has(p.id));
        if (!ready.length) {
            const next =
Math.min(...ps.filter(p=>!done.has(p.id)).map(p=>
p.arrival));
            pushBlock(timeline, 'idle', t, next);
            t = next; continue;
        }
        ready.sort((a,b)=>
(highestPriority ? b.priority-a.priority : a.priority-
b.priority) ||
a.arrival-b.arrival || a.idx-b.idx
);
        const p = ready[0];
        timeline.push({ pid:p.id, start:t, end:t+p.burst });
        finish[p.id] = t + p.burst;
        t += p.burst;
        done.add(p.id); completed++;
    }

    return buildMetrics(procs, timeline, finish);
}

```

```
/* 5b. Priority (preemptive) — uses module-level
`higherPriority` */
```

```
function priorityPreemptive(procs) {
  const ps = clone(procs);
  const finish = {}, timeline = [];
  let t = 0, completed = 0;
  while (completed < ps.length) {
    const ready = ps.filter(p=>p.arrival<=t &&
p.remaining>0);
    if (!ready.length) {
      const next =
Math.min(...ps.filter(p=>p.remaining>0).map(p=>p
.arrival));
      pushBlock(timeline, 'idle', t, next);
      t = next; continue;
    }
    ready.sort((a,b)=>
(higherPriority ? b.priority-a.priority : a.priority-
b.priority) ||
a.remaining-b.remaining || a.arrival-b.arrival ||
a.idx-b.idx
);
    const p = ready[0];
    pushBlock(timeline, p.id, t, t+1);
    p.remaining--; t++;
    if (p.remaining === 0) { finish[p.id]=t;
completed++; }
  }
  return buildMetrics(procs, timeline, finish);
}
```

```
/* 6. Priority + Round Robin — uses module-level
`higherPriority`
```

```
(local const higherPriority removed — that was
the bug) */
```

```
function priorityRoundRobin(procs, tq) {
  /* — FIX: no longer redeclares higherPriority
here; uses module-level var — */
  const ps = clone(procs).sort((a,b)=>a.arrival-
b.arrival||a.idx-b.idx);
  const finish = {}, timeline = [];
  let t = 0, queue = [], enqueued = new Set(),
completed = 0;
```

```
function enqueueArrivals() {
  ps.forEach(p => {
    if (p.arrival <= t && !enqueued.has(p.id) &&
p.remaining > 0) {
      queue.push(p); enqueued.add(p.id);
    }
  });
}
```

```
function sortQueue() {
  queue.sort((a,b)=>
(higherPriority ? b.priority-a.priority : a.priority-
b.priority) ||
a.arrival-b.arrival
);
}
```

```
enqueueArrivals();
```

```
while (completed < ps.length) {
  if (!queue.length) {
    const next = ps.filter(p=>p.remaining>0 &&
!enqueued.has(p.id))
      .reduce((m,p)=>p.arrival<m?p.arrival
:m, Infinity);
    if (next === Infinity) break;
    pushBlock(timeline, 'idle', t, next);
    t = next; enqueueArrivals(); continue;
  }
```

```
sortQueue();
```

```
const p = queue.shift();
```

```
let sliceLeft = Math.min(tq, p.remaining);
```

```
while (sliceLeft > 0) {
  pushBlock(timeline, p.id, t, t+1);
  p.remaining--; t++; sliceLeft--;
  enqueueArrivals();
}
```

```
if (sliceLeft > 0) {
  const topPri = queue.reduce(
(m,q) => higherPriority
```

```

    ? (q.priority > m ? q.priority : m)
    : (q.priority < m ? q.priority : m),
    higherPriority ? -Infinity : Infinity
  );
  if (higherPriority ? topPri > p.priority : topPri < p.priority) break;
}
}

if (p.remaining === 0) { finish[p.id] = t; completed++; }
else { queue.push(p); }
}

return buildMetrics(procs, timeline, finish);
}

/*
=====
=====
=====

METRICS BUILDER
=====
=====
===== */

function buildMetrics(origProcs, timeline, finish) {
  const metrics = [];
  for (const p of origProcs) {
    const ct = finish[p.id];
    const tat = ct - p.arrival;
    const wt = tat - p.burst;

    metrics.push({ pid:p.id, arrival:p.arrival, burst:p.burst, completion:ct, turnaround:tat, waiting:wt });
  }

  const totalBurst = origProcs.reduce((s,p)=>s+p.burst, 0);
  const totalElapsed = Math.max(...timeline.map(b=>b.end)) - Math.min(...timeline.map(b=>b.start));

  const utilization = totalElapsed > 0 ? (totalBurst / totalElapsed * 100).toFixed(1) : '100.0';

  return { timeline, metrics, utilization };
}
```

```

/*
=====
=====
=====

  RUN higherPriority, before calling any algo
  =====
  =====
  ===== */

runBtn.addEventListener('click', () => {
  clearErr();

  const algo = algoSel.value;

  if (!algo) { showErr('Please select an algorithm'); return; }

  const tq = parseInt(tqInput.value);

  if (needsTQ(algo) && (!tq || tq < 1)) { showErr('Time Quantum must be ≥ 1'); return; }

  const procs = readProcesses();

  if (!procs.length || !procs.every(p=>p.burst>0)) { showErr('All burst times must be > 0'); return; }

  /* — FIX: read the dropdown once here so ALL algorithms see the same value — */

  higherPriority = priorityTypeSelect.value === 'higher';

  let res;

  try {
    if (algo==='FCFS') res = fcfs(procs);
    else if (algo==='SJF') res = sjf(procs);
    else if (algo==='SRT') res = srt(procs);

    else if (algo==='RR') res = roundRobin(procs, tq);

    else if (algo==='Priority') {
      const isPreemptive = preemptiveSelect.value === 'true';

      res = isPreemptive ? priorityPreemptive(procs) : priority(procs);
    }

    else if (algo==='PriorityRR') res = priorityRoundRobin(procs, tq);

    else { showErr('Unknown algorithm'); return; }
  } catch(e) { showErr('Error: '+e.message); return; }

  renderOutput(res, algo);
}
```

```
});

/*
=====
=====
=====

RESET
=====
=====
===== */

resetBtn.addEventListener('click', () => {
  clearErr();
  algoSel.value = "";
  countInput.value = "";
  tqInput.value = "";
  preemptiveSelect.value = 'false';
  tqRow.classList.remove('visible');
  preemptiveRow.style.display = 'none';
  procBody.innerHTML = "";
  outputPanel.innerHTML = `
    <div class="output-placeholder"
id="placeholder">
      <svg width="48" height="48" viewBox="0 0 48
48" fill="none">
        <rect x="4" y="20" width="8" height="20"
rx="2" fill="#8b9a92"/>
        <rect x="16" y="12" width="8" height="28"
rx="2" fill="#8b9a92"/>
        <rect x="28" y="6" width="8" height="34"
rx="2" fill="#8b9a92"/>
        <rect x="40" y="16" width="8" height="24"
rx="2" fill="#8b9a92"/>
      </svg>
      <p>Configure & run simulation</p>
    </div>`;
});

/*
=====
=====
=====

RENDER OUTPUT
=====
=====
===== */

function renderOutput(res, algo) {
```

```
const { timeline, metrics, utilization } = res;

const maxTime =
Math.max(...timeline.map(b=>b.end));

const ganttBlocks = timeline.map(b => {
  const cls = b.pid === 'idle' ? 'idle-block' :
PC[parseInt(b.pid.slice(1))-1];
  const dur = b.end - b.start;
  const tip = b.pid==='idle'
    ? `Idle [${b.start}-${b.end}] (${dur}
unit${dur!==1?'s':''})`
    : `${b.pid} [${b.start}-${b.end}] (${dur}
unit${dur!==1?'s':''})`;
  return `<div class="gantt-block ${cls}"
style="flex:${dur}" data-tip="${tip}">
    <span class="g-label">${b.pid}</span>
  </div>`;
}).join("");

const axisTicks =
Array.from({length:maxTime+1},(_i,i)=>
  `<div class="gantt-tick"
style="left:calc(${i}/(maxTime||1)*100)%
0px">${i}</div>`
).join("");

const avgWT =
(metrics.reduce((a,m)=>a+m.waiting,0)/metrics.len
gth).toFixed(2);
const avgTAT =
(metrics.reduce((a,m)=>a+m.turnaround,0)/metrics
.length).toFixed(2);

const html = `
  <div class="gantt-panel">
    <div class="gantt-scroll">
      <div class="gantt-
track">${ganttBlocks}</div>
      <div class="gantt-axis"
style="height:18px;">${axisTicks}</div>
    </div>
  </div>
  <div class="stats-row">
    <div class="stat-card">
      <div class="stat-label">Avg Waiting
Time</div>
```

